Query Performance Analysis

ProGineer Technical Assessment | Part 4 | MySQL | Selected queries: Q2, Q4, Q8, Q9 and Q20

1. Scope and methodology

This report reviews five queries from the submitted Q1-Q20 solution against the existing ECommerceDB schema. It covers access paths, join choices, indexing, duplicate prevention, growth risks, and execution-plan interpretation. Existing composite indexes include Orders(idCustomer, orderDate), Products(idVendor, name), Reviews(idProducts, rating) and PriceHistory(idProducts, changedAt).

Evidence boundary: No live MySQL execution plan was available when this report was prepared. The EXPLAIN descriptions below are expected plan characteristics, not observed access types, row counts, costs, or timings. The accompanying part4_explain.sql runs EXPLAIN FORMAT=JSON for these exact queries.

2. Selected query analysis

Q2 - Unordered Products

Purpose: Find products absent from OrderItems, while returning the owning vendor.

Access paths: Products is the outer relation; Vendors joins by its primary key. A NOT EXISTS anti-match probes OrderItems by productId. MySQL may implement this as an anti-join or subquery lookup.

Indexes: Products.id and Vendors.id are primary keys. OrderItems.productId requires an index; InnoDB normally creates or reuses a supporting index for this foreign key. Confirm with SHOW INDEX.

Duplicate control: NOT EXISTS is a boolean existence test, so multiple order lines cannot duplicate a product.

Scalability: With an index, work grows roughly with product count plus matching probes. Without one, repeated OrderItems scans are costly.

EXPLAIN analysis: Inspect the key chosen for OrderItems, access_type (ref or an equivalent anti-join plan), and estimated examined rows. A scan of Products is reasonable because every product must be tested.

Q4 - Top 10 Spending Customers

Purpose: Find the ten customers with the largest spending across Delivered orders.

Access paths: Customers joins Orders through idCustomer, then OrderItems through idOrder; Orders is filtered by status. MySQL can choose a different starting table based on selectivity.

Indexes: Existing Orders(idCustomer, orderDate) aids customer lookups but does not directly index status. Consider Orders(status, idCustomer) if Delivered filtering is selective; OrderItems has UNIQUE(idOrder, productId) supporting order-line joins.

Duplicate control: One order expands to many items. COUNT(DISTINCT o.id) counts the order once, while SUM(quantity * unitPrice) intentionally counts every sold line.

Scalability: Large joins and grouping can create temporary aggregation structures and sorting. Pre-aggregated order totals may help high-volume recurring reports.

EXPLAIN analysis: Look for the Orders status filter, its chosen key, OrderItems ref lookups, grouping overhead, and temporary/filesort operations. LIMIT 10 does not avoid calculating each customer total.

Q8 - Frequently Bought Together

Purpose: Count distinct product pairs appearing in the same order.

Access paths: OrderItems is joined to itself on idOrder, with oi1.productId < oi2.productId. Both product names use primary-key lookups in Products.

Indexes: The existing UNIQUE(idOrder, productId) index supports finding order mates and matching by productId within an order. Product primary keys serve the name lookups.

Duplicate control: The strict less-than comparison returns (A,B) only, never (B,A) or (A,A). COUNT(DISTINCT oi1.idOrder) counts a pair at most once per order.

Scalability: An order with k distinct products creates k(k-1)/2 pairs. Large baskets cause quadratic work; aggregate co-purchases incrementally for frequent reporting.

EXPLAIN analysis: Look for an index range/ref access on the second OrderItems alias and primary-key lookups on Products. Grouping/distinct aggregation and result ordering can require temporary work.

Q9 - Month-over-Month Revenue Growth

Purpose: Sum Delivered-order line revenue by calendar month and compare each month with the preceding calendar month without LAG().

Access paths: Orders filtered to Delivered joins OrderItems by idOrder. MonthlyRevenue groups those rows by a derived month start. A left self-join matches the immediately previous calendar month.

Indexes: Orders(status, orderDate) is a possible additional composite index for the status/date access pattern. The existing UNIQUE(idOrder, productId) supports order-line retrieval. DATE_FORMAT on orderDate prevents direct index-ordered month grouping.

Duplicate control: One order can have several lines, but each line contributes once before monthly aggregation. MonthlyRevenue has one row per month, preventing inflated totals in the self-join.

Scalability: Grouping a growing full history may dominate cost. For recurring dashboards, maintain a monthly revenue summary; use date-range predicates to limit scans when allowed.

EXPLAIN analysis: Expect access to Orders and OrderItems, a grouping/materialization step for MonthlyRevenue, and a lookup or scan for the previous-month match. Missing months are not generated; growth is NULL when previous-month revenue is zero or absent.

Q20 - Vendor Performance Matrix

Purpose: Calculate revenue, completed-order count, units, average order value, average rating and unsold products for every vendor.

Access paths: ProductSales aggregates Products with OrderItems and Delivered Orders at product grain. VendorOrders counts vendor-specific completed orders. VendorRatings aggregates Reviews separately. These vendor-grain results are left-joined to Vendors.

Indexes: Existing Products(idVendor, name) and Reviews(idProducts, rating) help two join paths. OrderItems needs efficient access by productId and idOrder; the schema has supporting foreign-key and UNIQUE(idOrder, productId) indexes. Check their actual keys before adding any others.

Duplicate control: Sales, distinct order counts and ratings are aggregated separately to avoid multiplying sales lines by review rows. COUNT(DISTINCT o.id) protects against multiple items from one vendor in the same order.

Scalability: Multiple CTE aggregations can scan large transaction tables and create temporary grouped results. Incremental product/vendor summaries can reduce repeated cost for frequent reports.

EXPLAIN analysis: Inspect materialized/derived groups, scans of OrderItems and Reviews, COUNT(DISTINCT) work and final left joins. A full Vendors scan is acceptable because the report must include vendors with zero sales.

3. Execution-plan review

Run part4_explain.sql after loading the schema and sample data. Record access_type, key, rows_examined_per_scan, attached_condition, and any temporary-table or sort operations. An ALL scan is not automatically a problem for small datasets; assess it relative to table size and filtered rows. The optimizer may choose different access orders after data volumes and statistics change.

For measured runtimes on MySQL 8.0.18+, run EXPLAIN ANALYZE on each SELECT statement separately. Compare plans before and after any added index, using representative data volumes. The small seed dataset is not evidence of production-scale performance.

4. Indexing decisions and trade-offs

The two strongest candidates to evaluate, rather than apply blindly, are Orders(status, orderDate) for monthly Delivered-order filtering and OrderItems(productId, idOrder) for product-first reporting. Existing FK-support indexes may already cover part of these patterns. Composite index prefix order matters, and each additional index consumes storage and increases write cost. Use observed EXPLAIN output before changing the schema.

5. Conclusion

Q2 benefits from efficient anti-match lookups; Q4 from filtered joins and correct DISTINCT counting; Q8 is limited by product-pair growth; Q9 by repeated historical aggregation; and Q20 by several large but intentionally separated aggregates. The recommended improvements target these costs without changing the queries' business meaning.